Logiciels Sûrs

Allel HADJALI

Professeur en Informatique

LIAS/ISAE-ENSMA - Poitiers

allel.hadjali@ensma.fr

GROUPE isae
Pôle mondial de formation & de recherche en aéronautique et spatial

Université de Poitiers

Répartition du Volume Horaire

- **Cours : 10h**
  - **A. HADJALI**
- **TD : 10h**
  - **A. HADJALI**
- **Evaluation : Examen écrit**

Contenu

- **Chapitre 0** : Principes du génie logiciel
- **Chapitre 1** : Outillage formel
- **Chapitre 2** : Développement de logiciels sûrs
- **Chapitre 3** : Vérification formelle de logiciels
- **Chapitre 4** : Model-Checking & Logiques temporelles
- **Chapitre 5** : Tests logiciels

Références Recommandées

- Jean-Raymond Abrial, The B Book - Assigning Programs to Meanings Cambridge University Press, August 1996.
- Steve Schneider, The B Method: An Introduction, CornerStones of Computing, 2001.
- Jean-Raymond Abrial, *Modeling in Event-B: System and Software Engineering, 2010*
- A-P. Mathur, Foundations of Software Testing. Pearson Education, 2008.
- C. Baier and J-P. Katoen. Principles of model checking, MIT Press, 2008.
- C.A. Gunter and D.S. Scott, Semantic Domains, In Jan van Leeuwen, Editor, Handbook of theoretical computer science, Elsevier Science, Publisher, 1990 (pp. 633-676).
- Jacques Julliand, Vérifier, tester et concevoir des programmes en les modélisant, Vuibert, 2010 | 272 pages | 9782311000207
- E-M., Clarke, T-A., Henzinger, H. Veith, R. Bloem, Handbook of Model Checking, Springer, 1st ed. 2018 edition (May 18, 2018).

Principes du génie logiciel

LOS

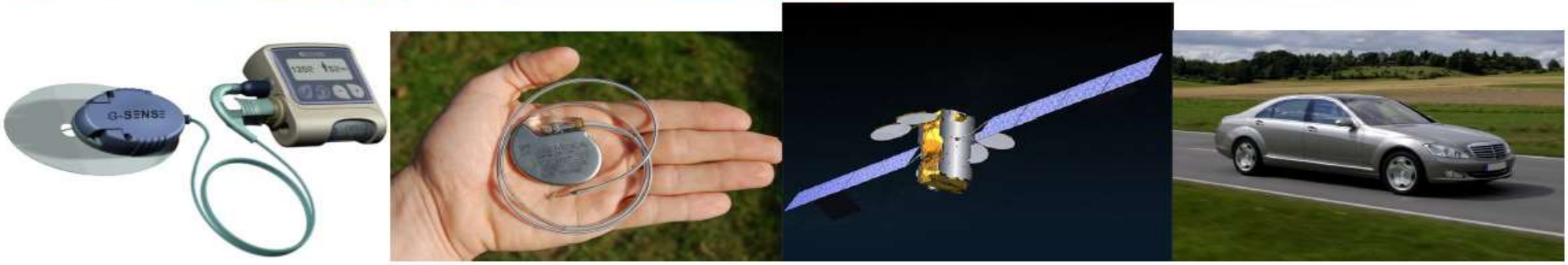
Chapitre 1

Détails du Chapitre I

- Introduction
- Qualités attendues d'un logiciel
- Processus de développement

Introduction

Omniprésence du logiciel ... et de ses défauts



Introduction

➔ Matériel et logiciel

- Système informatiques
  - 80 % de logiciel
  - 20 % de matériel
- Depuis quelques années, la fabrication du matériel est assurée par quelques fabricants seulement
  - Le matériel est relativement fiable
  - Le marché est standardisé
- Les problèmes liés à l'informatique sont essentiellement des problèmes de **Logiciel**

Introduction

➔ Spécificités du logiciel

- Un produit immatériel, dont l'existence est indépendante du support physique
  - Semblable à une œuvre d'art (roman, partition, …)
- Un objet technique fortement contraint
  - Fonctionne ou ne fonctionne pas
  - Structure complexe
  - Relève des modes de travail du domaine technique
- Un cycle de production différent
  - La reproduction pose peu de problèmes, seule la première copie d'un logiciel a un coût
  - Production à l'unité
  - Semblable au génie Civil (ponts, routes, …)

Introduction

➔ La « Crise du logiciel »

- Etude sur 8380 projets (*Standish Group, 1995*)
  - Succès : 16 %
  - Problématique : 53 % (budget ou délais non respectés, défaut de fonctionnalités)
  - Echec : 31 % (abandonné)
- Le taux de succès décroit avec la taille des projets et la taille des entreprises

Introduction

- Conférence de l'OTAN à Garmish, Allemagne (1968) a approuvé les constatations :
  - Le logiciel développé n'était pas fiable
  - L'informatique ne répond pas aux attentes qu'elle suscite
  - Il était devenu incroyablement difficile de réaliser dans des délais prévus des logiciels répondant aux cahiers des charges

- Naissance de l'expression « Génie Logiciel » (Software Engineering) :
  - Comment faire des logiciels de qualité
  - Qu'attend-on d'un logiciel? Quels sont les critères de qualité pour un logiciel?

Introduction

➔ Erreurs célèbres :

- Sonde Mariner vers Vénus perdue dans l'espace : Erreur de programmation (FORTRAN)
- En 1981, le 1er lancement de la navette spatial a été retardé de 2 jours : un problème logiciel
- Explosion d'Ariane (1996) est due à une faute logicielle de l'une de ses composantes
- Développement du compilateur PL1 (chez Control Data) n'a jamais vu le jour
- SNCF a rencontré des difficultés importantes pour la mise en service du système Socrate
- Avion C17 de McDonnell Douglas livré avec un dépassement de 500 millions de $ ... (19 calculateurs hétérogènes et 6 langages de programmation différents),

Introduction

Echec du Vol Ariane 501

- Vol inaugural d'Ariane 5 qui a eu lie le 04 juin 96 s'est soldé par un échec
- Environ 40s seulement après le démarrage de la séquence de vol, le lanceur, qui se trouvait alors à une altitude de qlq 3700 mètres, a dévié de sa trajectoire, s'est brisé et a explosé.
- Des ingénieurs des équipes du projet Ariane 5 du CNES et de l'industrie ont immédiatement commencé à **rechercher les causes de cet échec**.

Introduction

Echec du Vol Ariane 501

1996 Lancement du Satellite Ariane 5 (vol 501)

FIG. : Ariane 5G V88/501, 4 Juin 1996 12 :34 :43 UTC

Introduction

Ariane 501 : Commission d'enquête

- Déterminer les causes de l'échec du lancement
- Analyser l'adéquation des essais de qualification et des essais de recette face aux problèmes rencontrés.
- Recommander les actions correctives pour éliminer les causes de l'anomalie et d'autres faiblesses éventuelles des systèmes incriminés.

Introduction

Ariane 501 : Cause de l'accident

- Perte total des informations de guidage et d'attitude 37s après le démarrage de la séquence d'allumage du moteur principal.
- Cette perte est due à des erreurs de spécification et de conception logiciel du système de référence inertielle.
  - Mauvaise transmission de données à cause d'une erreur d'opérande trop élevé
  - Conversion d'un flottant sur 64 bits en un entier signé sur 16 bits ……. La valeur du flottant dépassait la valeur maximale pouvant être convertie

Introduction

Contrôleur de vitesse des voitures Toyota

En 2005, une accélération soudaine de voiture sur une autoroute fait un mort.

En 2013, une cour de justice d'Oklahoma condamne Toyota à 1.5M$ de dommages. . .

Entre temps, de nombreux autres accidents, au moins 89 morts attribuées. [4]

4. CBSNews. Toyota "Unintended Acceleration" Has Killed 89. 20 March 2014.

Introduction

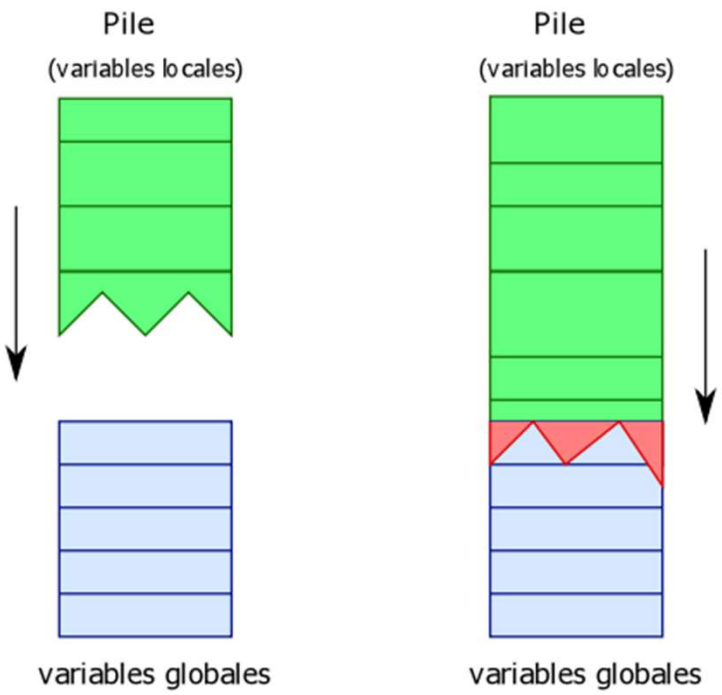
Contrôleur de vitesse des voitures Toyota : Explication

Coupable : logiciel de contrôle de vitesse.

Une première étude, de la NASA, dure 10 mois et est inconclusive.
Une deuxième étude, de Michael Barr, conclue que le code est de mauvaise qualité.

Exemple de défaut :
une pile trop petite pouvant causer une corruption mémoire, soupçonnée de causer le bug.

Toyota avait effectué une analyse statique de la pile sur son logiciel, mais celle-ci aurait ignoré certains appels à des bibliothèques et appels par pointeur de fonction, sous-estimant la taille de pile utilisée



Introduction

- D'où le besoin pour le développement de logiciel de s'appuyer sur des bases théoriques et un ensemble de méthodes (rigoureuses) et outils validés par la pratique (comme c'est le cas en génie électrique ou en génie civil, etc.)

- Le génie logiciel considère donc le logiciel comme un objet manufacturé complexe
  - Domaine des « Sciences de l'ingénieur » dont la finalité est la *conception*, la *fabrication* et la *maintenance de systèmes logiciels complexes, sûrs et de qualité.*
  - Son but est de définir des techniques de « fabrication » justifiées soit par la théorie, soit par la pratique (Simulation)

- Le génie logiciel est donc l'art de spécifier, de concevoir, de réaliser, et de faire évoluer, avec des moyens et dans des délais raisonnables, des programmes de qualité en vue d'utiliser un ordinateur pour résoudre certains problèmes.

Introduction

- Comme tout produit manufacturé complexe, un logiciel est produit en suivant un certain processus.
- Le Développement du Logiciel (DL) est une suite de descriptions de plus en plus précises et de plus en plus proches d'un programme exécutable.
  - Les passages d'une description à une autre sont appelées des raffinements successifs
- Caractéristiques du processus DL
  - Nature itérative : Certains étapes déclenchant la révision du résultat des étapes précédentes
  - Invisibilité : Difficile d'observer un logiciel en cours de développement et de dire que le processus se déroule correctement
  - Maintenance : Corriger des défauts, améliorer certaines caractéristiques, répondre à des évolutions de besoins, etc.
- Modèles DL :
  - Développement en étapes (phases) : Chaque étape se termine par la production de documents utile pour l'étape suivante
  - Différentes Modèles : Modèle en V, en spirale, etc.

Qualités attendues d’un logiciel

Qualités du Logiciel

- Utilité
  - Adéquation entre
    - Le besoin effectif de l'utilisateur
    - Les fonctions offertes par le logiciel
  - Solutions :
    - Emphase sur l'analyse des besoins
    - Améliorer la communication (langage commun, démarche participative)
    - Travailler avec rigueur

Qualités du Logiciel

➔ Utilisabilité

- Facilite d'apprentissage : comprendre ce que l'on peut faire avec le logiciel, et savoir comment le faire
- Facilite d'utilisation : importance de l'effort nécessaire pour utiliser le logiciel a des fins données
- Solutions :
  - Analyse du mode opératoire des utilisateurs
  - Adapter l'ergonomie des logiciels aux utilisateurs

Qualités du Logiciel

Fiabilité

- Correction, justesse, conformité : le logiciel est conforme à ses spéciations, les résultats sont ceux attendus
- Robustesse, sureté : le logiciel fonctionne raisonnablement en toutes circonstances, rien de catastrophique ne peut survenir, même en dehors des conditions d'utilisation prévues
- Mesures :
  - MTBF : Mean Time Between Failures
  - Disponibilité (pourcentage du temps pendant lequel le système est utilisable) et Taux d'erreur (nombre d'erreurs par KLOC)
- Solutions :
  - Utiliser des méthodes formelles, des langages et des méthodes de programmation de haut niveau
  - Vérifications, tests
  - Progiciels

Qualités du Logiciel

➔ Interopérabilité / Couplabilité

- Un logiciel doit pouvoir interagir en synergie avec d'autres logiciel
- Solutions :
  - Bases de données (découplage données / traitements)
  - « Externaliser » certaines fonctions en utilisant des « Middleware » avec une API (Application Program Interface) bien définie
  - Standardisation des formats de fichiers (XML ...) et des protocoles de communication (CORBA ...)

Qualités du Logiciel

- Performance
  - Les logiciels doivent satisfaire aux contraintes de temps d'exécution
  - Solutions :
    - Logiciels plus simples
    - Veiller à la complexité des algorithmes
    - Machines plus performantes

Qualités du Logiciel

- Portabilité
  - Un même logiciel doit pouvoir fonctionner sur plusieurs Machines / Environnements
  - Solutions :
    - Rendre le logiciel indépendant de son environnement d'exécution (voir interopérabilité)
    - Machines virtuelles

Qualités du Logiciel

- Réutilisabilité
  - On peut espérer des gains considérables car dans la plupart des logiciels :
    - 80 % du code est du « tout venant » qu'on retrouve a peu près partout
    - 20 % du code est spécifique
  - Solutions :
    - Abstraction, généricité
    - Construire un logiciel a partir de composants prêts à l'emploi
    - « Design Patterns »

Qualités du Logiciel

➔ Facilité de maintenance

- Un logiciel ne s'use pas (il devient obsolète par rapport aux concurrents, par rapport au contexte technique, par rapport aux autres logiciels, ...),
- Pourtant, la maintenance absorbe une très grosse partie des efforts de développement

	Répartition effort dév.	Origine des erreurs	Coût de la maintenance
Définition des besoins	6%	56%	82%
Conception	5%	27%	13%
Codage	7%	7%	1%
Intégration Tests	15%	10%	4%
Maintenance	67%		

(Zeltovitz, De Marco)

Qualités du Logiciel

- Maintenance corrective
  - Corriger les erreurs : défauts d'utilité, d'utilisabilité, de fiabilité...
    - Identifier la défaillance, le fonctionnement
    - Localiser la partie du code responsable
    - Corriger et estimer l'impact d'une modification
  - Attention :
    - La plupart des corrections introduisent de nouvelles erreurs
    - Les coûts de correction augmentent exponentiellement avec le délai de détection
  - La maintenance corrective donne lieu a de nouvelles livraisons (release)

Qualités du Logiciel

- Maintenance adaptative
  - Ajuster le logiciel pour qu'il continue a remplir son rôle compte tenu du l'évolution des
    - Environnements d'exécution
    - Fonctions à satisfaire
    - Conditions d'utilisation
  - Ex : changement de SGBD, de machine, de taux de TVA, an 2000, euro...

Qualités du Logiciel

- Maintenance perfective, d’extension
  - Accroitre/améliorer les possibilités du logiciel
  - Ex : les services offerts, l'interface utilisateur, les performances...
  - Donne lieu a de nouvelles versions

Qualités du Logiciel

Facilité de maintenance

- Objectifs
  - Réduire la quantité de maintenance corrective (zéro défaut)
  - Rendre moins couteuses les autres maintenances
- Enjeux :
  - Les coûts de maintenance se jouent très tôt dans le processus d'élaboration du logiciel
  - Au fur et à mesure de la dégradation de la structure, la maintenance devient de plus en plus difficile
- Solutions :
  - Réutilisabilité, modularité
  - Vérifier, tester
  - Structures de données complexes et algorithmes simples
  - Anticiper les changements a venir
  - Progiciels

Qualités du Logiciel

➔ Attention

Ces qualités sont parfois *contradictoires* (chic et pas cher !). Il faut les *pondérer* selon les circonstances (logiciel critique / logiciel grand public).

Il faut aussi distinguer les systèmes sur mesure et les produits logiciels de grande diffusion

Processus de développement

Processus de développement : Présentation

« La qualité du processus de fabrication est garante de la qualité du produit »

➔ Pour obtenir un logiciel de qualité, il faut en maitriser le processus d'élaboration

- La vie d'un logiciel est composée de différentes étapes
- La succession de ces étapes forme le cycle de vie du logiciel
- Il faut contrôler la succession de ces différentes étapes

Processus de développement : Présentation

Processus de développement logiciel : désigne l'ensemble des activités nécessaires au développement et à la maintenance d'un logiciel

➔ **Classiquement, on retrouve les activités suivantes :**

- **Analyse : "que faut-il faire" ou "ce qui doit être fait"**
  - **Expression des besoins :** ***comprendre et structurer les besoins du client***
  - **Spécification :** ***identifie les principales fonctionnalités et précise le contour du système à modéliser***
- **Conception : "comment faire ce qu'il faut faire"**

  Pour chaque besoin exprimé une solution informatique, manière dont le système doit être construit
- **Codage (Programmation) :**

  Réaliser le programme conformément aux critères définis dans les phases précédentes
- **Tests unitaires :**

  Vérifier individuellement la conformité de chaque élément du logiciel
- **Tests d'intégration :**

  Permet de vérifier l'assemblage des différents parties du logiciel
- **Test fonctionnels (Validation) :**

  Réalisée par le client pour s'assurer que l'application correspond bien au besoin qu'il a initialement exprimé

Processus de développement : Présentation

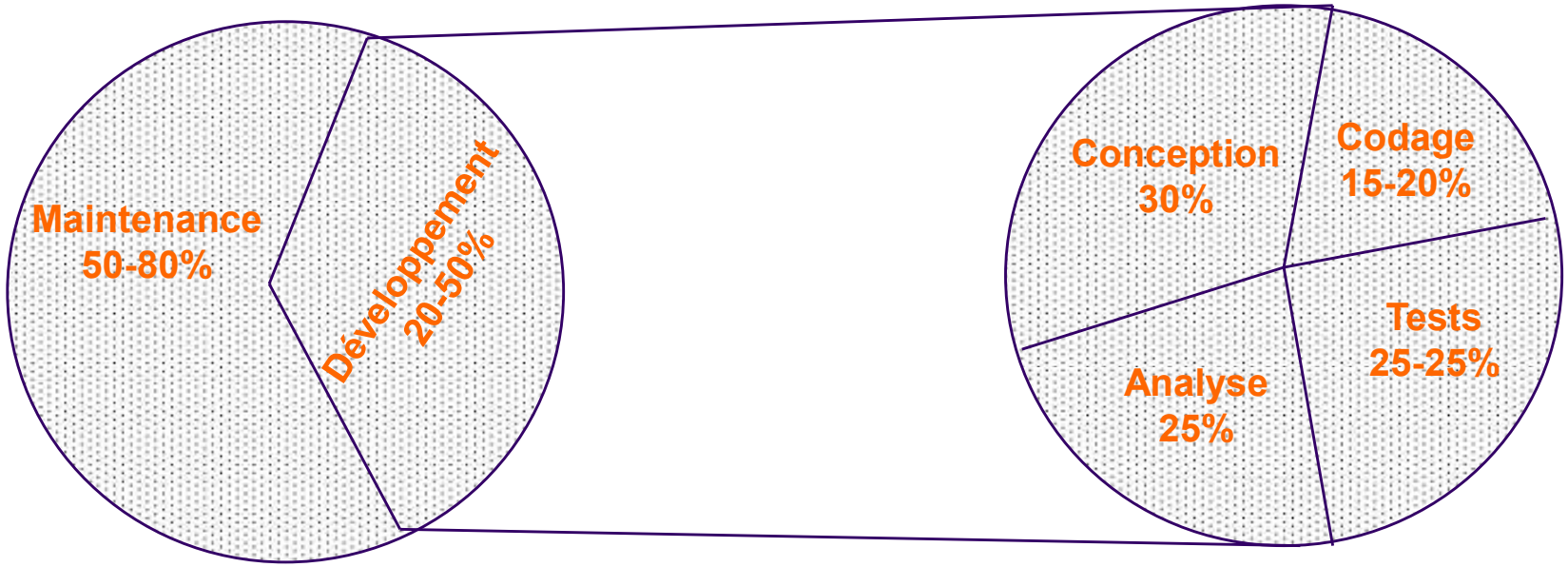
- **Analyse**

  Besoins + Spécifications = Analyse

- **Elle précise le "QUOI" (quoi faire?)**

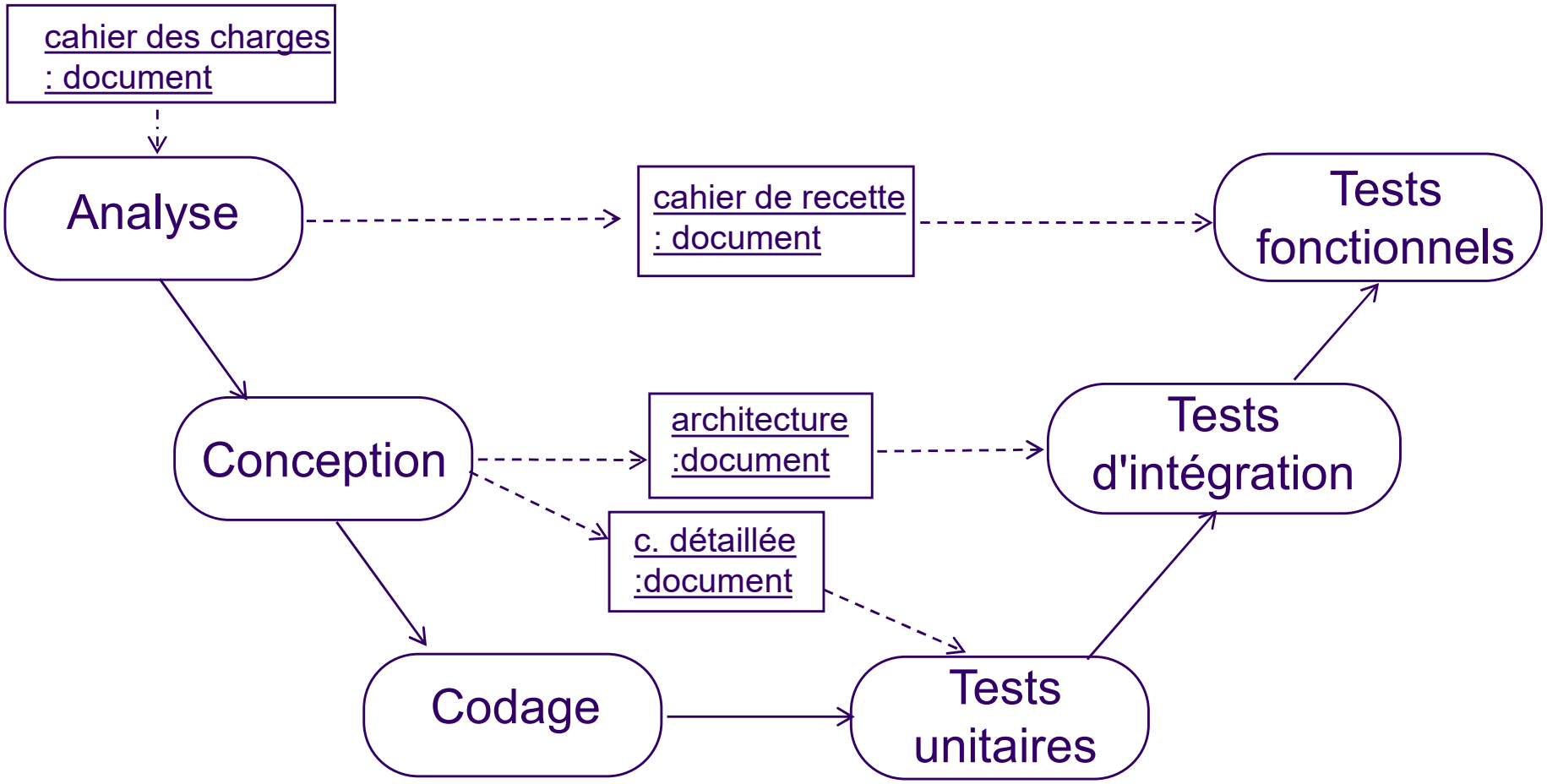
Pour être certain qu'un programme fasse exactement ce que l'on veut, il faut d'abord dire exactement ce que l'on veut qu'il fasse

Processus de développement : Proportions



Cycle de vie du logiciel

➔ **Modèle en V : le plus utilisé**



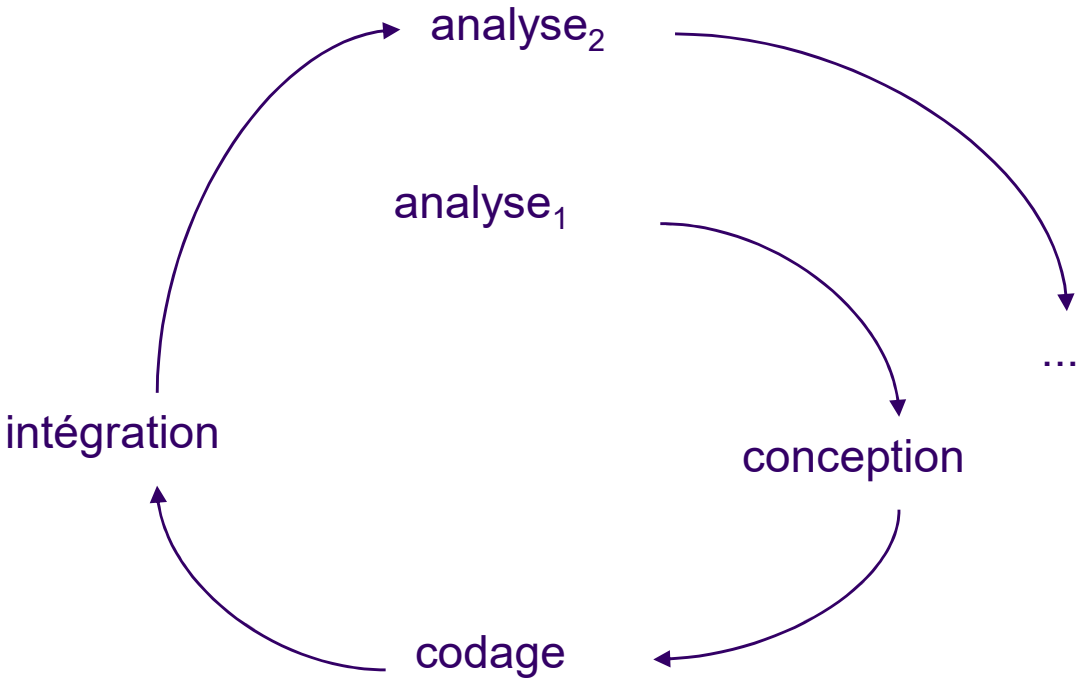
➔ ***Analyse*** **:** ***- En pratique, de nombreux retours arrières***

- Le client ne voit le produit que lorsque celui-ci est terminé (avec tous les risques …)

Cycle de vie du logiciel

➔ **Modèle en spirale : tentative de répondre aux lacunes de validation du modèle en V**

- **"Design a little, code a little"**
- **S'appuie sur une succession de cycles (plus court possible)**
- **Permet de fournit le plus rapidement possible un prototype**
- **A chaque itération, le logiciel est dans un état quasi-commercialisable**

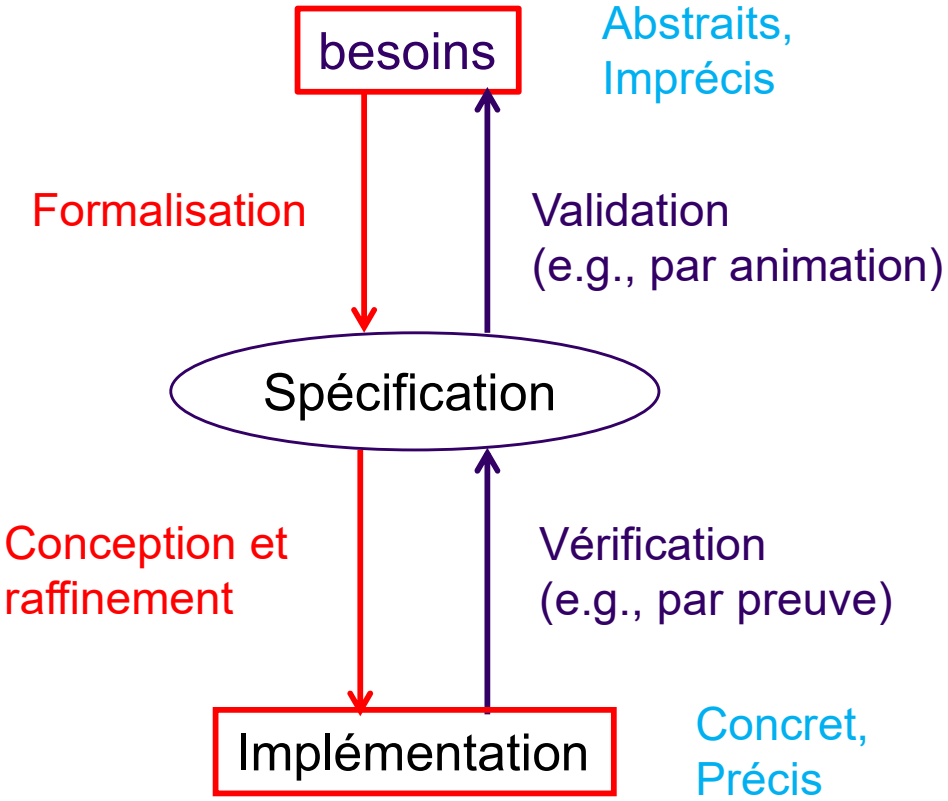


➔ ***Analyse*** **:**

- **grand intérêt en prototypage incrémental**
- **Très utilisé sur les projets reposant sur l'objet**
- **En cas de glissement de délais, il garantit une application qui tourne, bien que …..**

Spécification

- Spécification décrit d'une manière précise les exigences fonctionnelles et le comportement du logiciel
- Spécification formelle d'un logiciel = description précise mais abstraite de ce que le logiciel doit faire.
- Elle produit une étape intermédiaire entre les besoins (abstraits et imprécis) et le code exécutable (concret et précis)



Spécification

- **Spécifications doivent être claires, non ambiguës et compréhensibles**
- **Elles doivent aussi être cohérentes (pas de contradictions) et complètes (tous les concepts utilisés doivent être spécifiés)**
- **Formalismes existants :**
  - Spécifications informelles : langue naturelle (sans contraintes de forme)
  - Spécifications semi-formelles : notations graphiques ou semi-formelles (UML, SA-RT)
  - Spécifications formelles : syntaxe et sémantique sont définies formellement par des outils mathématiques

Spécification formelle (SF) : Une spécification est dite formelle si :

- **Elle est écrite en suivant une syntaxe bien définie, comme celle d'un langage de programmation;**
- **la syntaxe est accompagnée d'une sémantique rigoureuse qui définit des modèles mathématiques représentant la réalisation acceptable de chaque spécification syntaxique correcte.**

De plus, une SF permet des raisonnements : la syntaxe et la sémantique sont donc accompagnées de règles de déduction qui permettent de démontrer des propriétés d'une spécification.

Spécification

- **La syntaxe formelle concerne la manière dont sont construites les phrases d'un langage L**
  - La syntaxe indique les constructions licites du langage, c'est-à-dire quelles sont les expressions, mots-clés, structures autorisées du langage
- **La sémantique concerne la manière dont ces phrases peuvent être interprétées du point de vue sens**
- **Exemples de méthodes formelles (orientées modèles à états) :**
  - **Formalisme Z**
  - **Formalisme B : Du quoi au comment**
    - **Spécification et modélisation formelles**
    - **Permet le développement de programmes corrects par construction**

Sémantique formelle

- **Une ou plusieurs sémantiques qui associent un sens (si possible rigoureux au sens mathématique du terme) aux constructions du langages.**

- **La sémantique du langage donne une explication mathématique de l'exécution de tout programme :**
  - Quelle valeur nous décidons de donner à une expression ?
  - Quel effet nous décidons d'attribuer à une instruction ?

- **La sémantique permet de garantir certaines propriétés des programmes et raisonner sur ces propriétés :**
  - Equivalence de programmes : utile pour transformer un programme en un programme équivalent mais plus efficace
  - Terminaison de programmes

Différentes sémantiques formelles existent dans la littérature. Elles sont dites opérationnelle, dénotationnelle, axiomatique et ne sont pas nécessairement équivalentes.

Tests

- **Essayer le logiciel sur des données d'exemple pour s'assurer qu'il fonctionne correctement**

  - Tests unitaires : faire tester les parties du logiciel par leurs développeurs
  - Tests d'intégration : tester pendant l'intégration
  - Tests de validation : pour acceptation par l'acheteur

  - Tests système : tester dans un environnement proche de l'environnement de production
  - Tests Alpha : faire tester par le client sur le site de développement
  - Tests Béta : faire tester par le client sur le site de production

  - Tests de régression : enregistrer les résultats des tests et les comparer a ceux des anciennes versions pour vérifier si la nouvelle n'en a pas dégrade d'autres

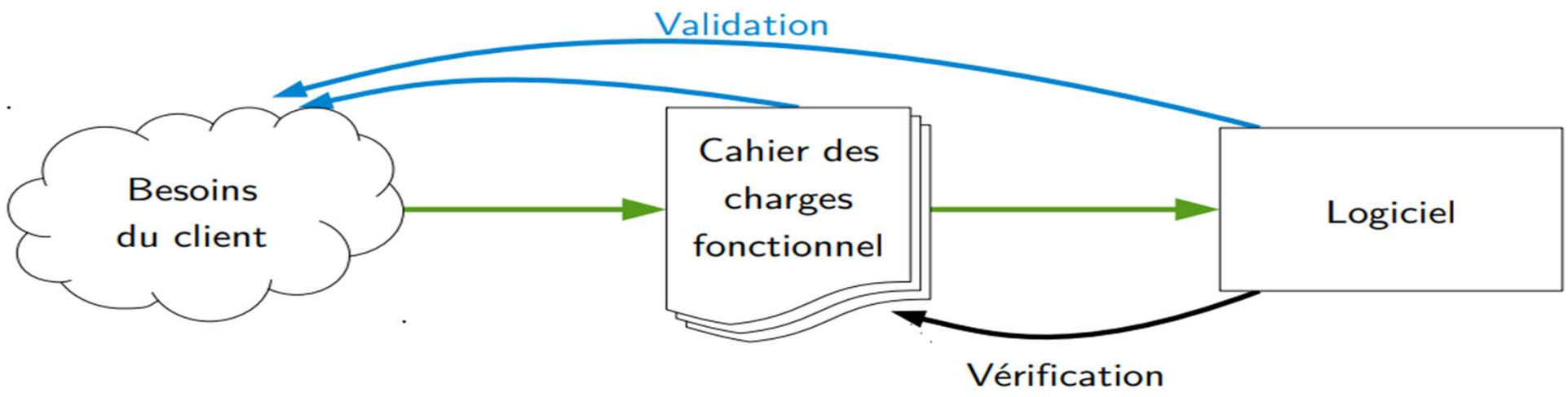
Nécessité d'une étape de VVT (Validation, Vérification & Test)

➔ **Comment effectuer de telles vérifications :**

- Test :
  - Nécessaire : permet de découvrir des erreurs
  - Pas suffisant : non exhaustif (prouve la présence d'erreurs, pas leur absence !)
- Vérification formelle / automatique :
  - Exhaustive
  - mise en œuvre difficile
- Model-Checking :
  - Exhaustif, partiellement automatique…
  - Mise en œuvre moins difficile (modèle formel + formalisation des propriétés)

Nécessité de VVT (Validation, Vérification & Test)

- **Validation :**
  - « *are we building the right product* » (« Construisons nous le bon produit ? »
  - Adaptation vis à vis des besoins des utilisateurs
  - Concerne les utilisateurs
- **Vérification :**
  - « *are we building the product right* » (« Construisons nous le produit correctement ? »
  - Correction interne du logiciel
  - Concerne les développeurs



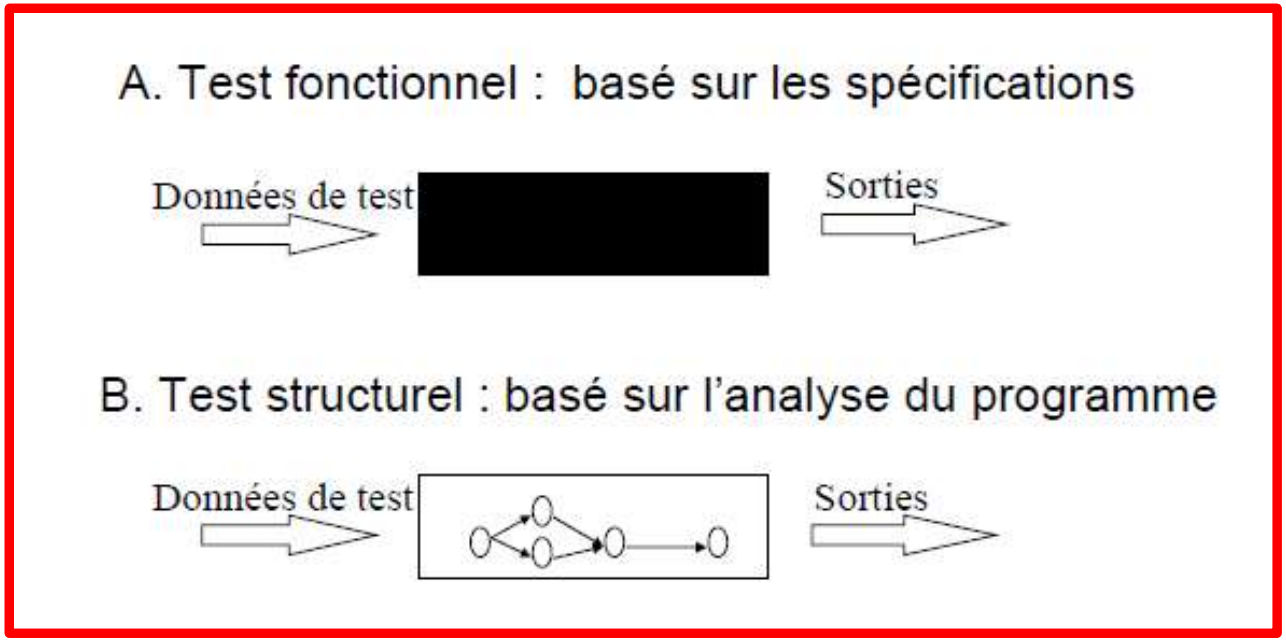
Nécessité de VVT (Validation, Vérification & Test)

Testing :

- « Evaluating software by observing its execution »

«Program Testing can be used to prove the presence of bugs, but never their absence » [Dijkstra 74]

Typology



Méthodes Formelles …

- **Savoir écrire des spécifications à l'aide de formules logiques**
  - *Modèle de données cohérent*
  - *Propriétés, invariant du système*
- **Savoir utiliser des outils automatiques démontrant des propriétés relatives à des programmes (correction, terminaison, sureté, …)**
  - *Correction, sûreté, …*
  - *Terminaison, équivalence, …*
  - *Validation & Vérification*

MF v.s. Simulation & Test (S&T)

- **S&T explorent certains comportements et scénarios possibles d'un système**
  - Ils laissent le problème de bugs possibles ouvert dans les cas des scenarios non explorés
- **FMs conduisent une exploration exhaustive de tous les comportements possibles en partant d'une spécification formelle**
  - Quand un système est déclaré correct par une FM, nous pouvons garantir que le système ne contient aucun bug

But du cours …

Recours aux Méthodes Formelles : Pourquoi

- Développer des logiciels / systèmes sûrs, fiables, robustes
- Inévitable dans les domaines dits critiques
  - Systèmes de transport (aéronautique, espace, ferroviaire, …)
  - Domaine de l'énergie (nucléaire, …)
  - Domaine militaire (lance missiles, …)
- Logiciels à fortes contraintes
  - Temps réel
  - Sécuritaire

Cas de l'Avionique ...

Avionique :

- domaine très critique ;
- les avions doivent être certifiés selon un processus strict ;
- encadré par les autorités de certification.

Pour la partie logicielle, le document guidant le processus est : DO-178 *Software Considerations in Airborne Systems and Equipment Certification.*

Évolution du document :

- révision DO-178B, 1992 : test, lecture de code, analyse ;
- révision DO-178C, 2012 : possibilité d'utiliser des méthodes formelles
  - sur le source, pour remplacer la lecture de code ;
  - sur le binaire, pour remplacer les tests ;

  $\Longrightarrow$ les méthodes formelles deviennent économiquement viables !

Exemple : chez Airbus, utilisation d'outils formels :

- Caveat (CEA) : méthode déductive, pour remplacer le test unitaire ;
- StackAnalyzer (AbsInt) : analyse statique de la pile ;
- Astrée (ENS, CNRS, AbsInt) : analyse statique des erreurs à l'exécution.